

Hochschule Hamm–Lippstadt

Priority Ceiling Protocol (PCP)

Resource Access Protocols in Fixed-Priority Real-Time Systems

Final milestone	Milestone 4: Scientific Communication — Paper and Talk
Student	Daniel Chidi Chimezie
Matriculation number	1230515
Group / Team	Group A / Team 9
Course	Real-Time Systems in Embedded Systems A Option
Supervisors	Prof. Dr. Stefan Henkler Prof. Dr. Achim Rettberg
Main reference	G. C. Buttazzo, <i>Hard Real-Time Computing Systems</i>
Submission date	23 June 2026

The author is responsible for the technical analysis, verification, citations, and final content of this submission.

TABLE OF CONTENTS

Abstract and I. Introduction	3
II. System Model and Priority Ceiling Protocol	3
III. Formal Properties and Blocking Analysis	3
IV. UPPAAL Implementation and Application Example	4
V. Verification Results and Critical Evaluation	4
VI. Conclusion and References	5

LIST OF FIGURES

Fig. 1. UPPAAL task-state structure for PCP	4
---	---

LIST OF TABLES

Table I. PCP notation	3
Table II. UPPAAL verification results	4
Table III. Resource-access protocol comparison	5

ABBREVIATIONS

PCP	Priority Ceiling Protocol
PIP	Priority Inheritance Protocol
SRP	Stack Resource Policy
WCET	Worst-Case Execution Time
RTA	Response-Time Analysis
EDF	Earliest Deadline First

Priority Ceiling Protocol for Predictable Resource Access

Daniel Chidi Chimezie

Department of Electronic Engineering, Hochschule Hamm-Lippstadt, Germany

Abstract

Shared resources can invalidate the predictability of fixed-priority real-time schedules because a high-priority task may wait for a lower-priority lock holder. This paper formalizes the Priority Ceiling Protocol (PCP), implements its central admission rule in UPPAAL, and evaluates ten safety and reachability properties. The case study contains three tasks and two resources whose ceilings are derived from the set of possible users. Every supplied query is satisfied: the model is deadlock-free, mutual exclusion and ownership remain consistent, and each task can reach its critical section. The analysis shows how PCP converts potentially chained or circular blocking into a computable single-critical-section bound, while also identifying configuration effort, conservative early blocking, and implementation overhead as practical costs.

Index Terms

priority ceiling protocol, priority inversion, real-time systems, resource sharing, UPPAAL, schedulability analysis

I. INTRODUCTION

A hard real-time computation is correct only if both its result and completion time satisfy the specification. Fixed-priority preemptive scheduling normally dispatches the highest-priority ready task. Mutual exclusion introduces a cross-priority dependency: a high-priority task can wait while a lower-priority task completes a critical section. If an unrelated medium-priority task preempts that lock holder, the high-priority task experiences extended priority inversion [1], [2]. Ordinary mutexes preserve data consistency but do not provide the blocking bound required by schedulability analysis.

The basic Priority Inheritance Protocol (PIP) raises the effective priority of a lock holder when it blocks a higher-priority task. This prevents unrelated medium-priority interference, but nested requests may still cause transitive inheritance, chained blocking, and deadlock. PCP adds an admission test based on static resource ceilings, preventing unsafe requests before circular waiting can form [2]. This paper states the protocol formally, demonstrates its mechanism in UPPAAL, and separates properties proved for the model from deployment claims that still require platform measurements.

II. SYSTEM MODEL AND PCP

A. Task and Resource Model

Let $\Gamma = \{\tau_1, \dots, \tau_n\}$ be periodic or sporadic tasks scheduled preemptively by fixed base priorities P_i , where a larger value denotes a higher priority. Task τ_i has worst-case execution time C_i , period or minimum inter-arrival time T_i , relative deadline D_i , and a set σ_i of binary semaphores it may lock. Critical sections are assumed to be properly nested and to have bounded duration. Table I summarizes the notation.

B. Resource and System Ceilings

Every resource receives the highest priority of any task that may use it:

$$C(S_k) = \max\{P_i \mid S_k \in \sigma_i\}. \quad (1)$$

TABLE I
PCP NOTATION, ADAPTED FROM [1], [2]

Symbol	Meaning
P_i	Base priority of task τ_i
$C(S_k)$	Static priority ceiling of resource S_k
$\Pi(t)$	Current system ceiling at time t
B_i	Worst-case lower-priority blocking of τ_i
R_i	Worst-case response time of τ_i

At time t , the system ceiling is the largest ceiling among resources locked by tasks other than the requester:

$$\Pi(t) = \max\{C(S_k) \mid S_k \text{ is locked}\}, \quad (2)$$

with $\Pi(t) = -\infty$ if the set is empty. A request $\text{lock}(\tau_i, S_k)$ is admitted only if S_k is free and $P_i > \Pi(t)$. A denied requester blocks. The task responsible for the ceiling inherits the blocked task's priority until the relevant resource is released [1], [2].

C. Operational Algorithm

The main protocol mechanism is:

- 1) Offline, enumerate every possible task–resource relation and compute $C(S_k)$ using (1).
- 2) On a lock request, determine the current owner of S_k and compute $\Pi(t)$ excluding resources already owned by the requester.
- 3) Grant the request exactly when the resource is free and $P_i > \Pi(t)$; atomically update its lock and owner state.
- 4) Otherwise block τ_i and propagate its priority to the task responsible for the blocking ceiling.
- 5) On unlock, clear ownership, recompute the system ceiling and effective priority, wake eligible tasks, and dispatch the highest-priority ready task.

III. FORMAL PROPERTIES AND BLOCKING ANALYSIS

A. Deadlock Prevention and Bounded Blocking

The ceiling test prevents a task from acquiring a resource when a resource held elsewhere already indicates a possible

future conflict at an equal or higher priority. Therefore the unsafe edge needed to complete a circular-wait chain is not admitted. Under the standard uniprocessor PCP assumptions, a job can be blocked by at most one lower-priority critical section [1], [2]. A conservative bound is

$$B_i = \max_{\substack{P_\ell < P_i \\ C(S_k) \geq P_i}} \{cs_{k,\ell}\}, \quad (3)$$

where $cs_{k,\ell}$ is the longest interval for which lower-priority task τ_ℓ holds S_k . Equation (3) bounds *lower-priority blocking*; it does not remove ordinary interference by higher-priority tasks.

B. Response-Time Analysis

For a fixed-priority task with a constrained deadline, blocking is inserted explicitly into the standard response-time recurrence:

$$R_i^{(q+1)} = C_i + B_i + \sum_{\tau_j \in hp(i)} \left\lceil \frac{R_i^{(q)}}{T_j} \right\rceil C_j. \quad (4)$$

Starting with $R_i^{(0)} = C_i + B_i$, the task is schedulable if the iteration converges to a fixed point $R_i \leq D_i$; it fails the test once an iterate exceeds D_i [1]. PCP is valuable here because it turns an otherwise uncontrolled delay into the finite term B_i .

C. Complexity and Implementation Overhead

Ceilings are computed offline. A direct implementation scans m resources to recompute the system ceiling, giving $O(m)$ lock/unlock overhead and $O(m)$ state for ceilings, owners, and lock flags. Maintaining counts per ceiling or a priority-ordered data structure can reduce common operations to $O(1)$ or $O(\log m)$. Priority inheritance can also cause ready-queue updates and context switches. Most importantly, the resource-use declaration is safety-critical: omitting a possible user may assign a ceiling that is too low and invalidate the analytical guarantee.

IV. UPPAAL IMPLEMENTATION AND APPLICATION

A. Model Structure

The UPPAAL model represents each task by a timed automaton with *Idle*, *Request*, *Critical*, and *Release* behaviour. Global arrays record `locked[r]` and `owner[r]`, while `ceiling[r]` contains the offline configuration. The Request-to-Critical guard evaluates resource availability and the system-ceiling condition. Admission updates ownership atomically; Release clears it.

The concrete topology uses high-priority T1 and low-priority T3 on R1, hence $C(R1) = \text{HIGH}$. Medium-priority T2 uses R2, hence $C(R2) = \text{MEDIUM}$. If T3 owns R1 when T1 requests it, T1 blocks and T3 inherits HIGH until release. After the bounded critical section, T3 releases R1, returns to LOW, and T1 becomes eligible to enter.

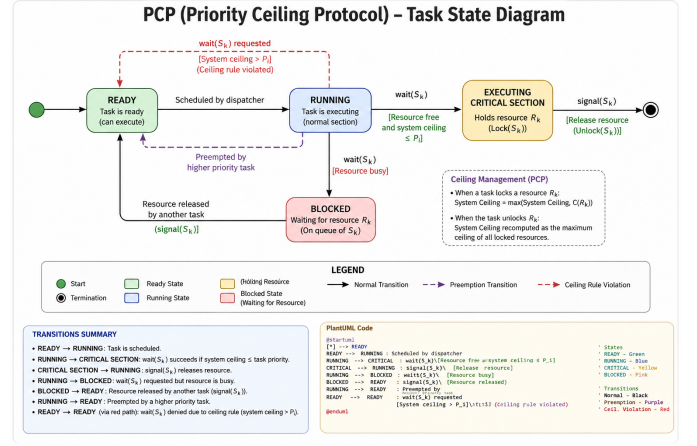


Fig. 1. UPPAAL task-state structure used for PCP admission and release. Technical basis adapted from [1]–[3]; colourful visual redesign assisted by ChatGPT Pro [4].

TABLE II
UPPAAL VERIFICATION RESULTS DERIVED FROM THE DESIGN
PARAMETERS AND VERIFICATION QUERIES OF THE AUTHOR'S PCP MODEL

Group	Representative query / count	Result
Deadlock	A[] not deadlock (1)	Satisfied
Mutual exclusion	Not(T1.Critical and T3.Critical) (1)	Satisfied
Ownership	Critical state implies correct owner (3)	Satisfied
Reachability	E<> each task Critical (3)	Satisfied
Lock consistency	Locked implies non-free owner (2)	Satisfied

B. Embedded-System Interpretation

The tasks can represent an embedded controller in which T1 performs emergency control, T3 performs low-priority diagnostics, and both share actuator bus R1; T2 performs logging through buffer R2. Without inheritance, logging may preempt diagnostics while emergency control waits. PCP allows diagnostics to complete the bounded bus transaction promptly and rejects resource sequences that could create circular waiting.

C. Verification Plan and Scope

The ten queries combine universal safety with existential reachability: deadlock freedom, R1 mutual exclusion, agreement between Critical locations and the owner array, lock consistency, and reachability of each task's critical section. Reachability is essential because safety alone could be obtained by a model that never admits useful work.

The model establishes logical PCP behaviour for this finite topology. It does not measure context-switch time, interrupt latency, cache effects, semaphore cost, or WCET. Those values must be measured or safely bounded on the target platform before (4) can support a deadline guarantee.

V. RESULTS AND CRITICAL EVALUATION

A. Verification Results

All ten supplied UPPAAL properties are satisfied. Table II groups the checks without hiding their individual role.

TABLE III
RESOURCE-ACCESS PROTOCOL COMPARISON, ADAPTED FROM [1]–[3]

Criterion	PIP	PCP	SRP
Deadlock prevention	Not inherent	Ceiling admission	Ceiling admission
Blocking	May chain	One lower CS	Bounded
Priority basis	Fixed	Fixed	Fixed / dynamic
Main cost	Inheritance	Configuration	Preemption levels

The universal invariants show that no encoded failure is reachable, while the existential properties show that every task can still perform useful critical work. The result is evidence for the supplied model and configuration, not a universal proof of every PCP implementation.

B. Comparison with Related Protocols

Table III places PCP between basic PIP and the Stack Resource Policy (SRP). SRP refines ceiling-based control and supports stack sharing and EDF-oriented analysis [3].

PCP is attractive when fixed priorities, nested locks, and analyzable blocking dominate the design. Its early blocking is deliberately conservative: a task may wait even when its requested resource is free because another locked resource exposes a future conflict. PIP is simpler conceptually but produces more complicated blocking chains. SRP is preferable when stack sharing or deadline-based scheduling is central [1], [3].

C. Strengths, Limitations, and Validity Threats

PCP prevents deadlock, excludes chained lower-priority blocking under its assumptions, and provides a blocking term that can be inserted directly into RTA. The executable model strengthens the argument by checking safety together with reachability.

The guarantees depend on complete prior knowledge of resource use and correct ceiling assignment. Dynamic tasks and resources complicate this assumption. Long critical sections still create long, although bounded, delays. UPPAAL abstracts operating-system and hardware overhead, and the verification is configuration-specific: a change in nesting, task priorities, or resource users requires a new ceiling audit and re-verification.

VI. CONCLUSION

PCP makes fixed-priority shared-resource behaviour predictable by combining static resource ceilings, admission control, and priority inheritance. In the three-task, two-resource UPPAAL model, all ten verification properties hold: the system is deadlock-free, resource ownership is consistent, mutual exclusion is preserved, and all tasks can reach their critical sections. Deployment should combine PCP with short critical sections, audited task–resource declarations, measured WCET and operating-system overhead, and response-time analysis.

Discussion Questions

When does conservative early blocking improve total predictability despite reducing immediate concurrency? How should ceilings be maintained when a software update adds a new resource user? Would SRP be preferable if the application moved from fixed-priority scheduling to EDF?

REFERENCES

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011, main course reference for task models, priority inversion, resource-access protocols, blocking analysis, and response-time analysis.
- [2] L. Sha, R. Rajkumar, and J. P. Lehoczky, “Priority inheritance protocols: An approach to real-time synchronization,” *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, Sep. 1990, foundational formal source for priority inheritance, PCP, deadlock prevention, and the single lower-priority critical-section blocking property.
- [3] T. P. Baker, “A stack-based resource allocation policy for realtime processes,” in *Proceedings of the 11th IEEE Real-Time Systems Symposium*, 1990, pp. 191–200, introduces the Stack Resource Policy used for the focused comparison with PCP.
- [4] OpenAI, “ChatGPT Pro,” Large language model service used only to assist the colourful visual redesign of Fig. 1, 2026, [Online]. Available: <https://chatgpt.com/>. Accessed: Jun. 23, 2026.